جلسهٔ ۲ از ۵

# ابزارهای عامل و تعامل‌پذیری

از کشف و پیکربندی تا اتصال

**بر پایهٔ مقالهٔ کانچانا پاتلولا، لوکاش اولینیچاک و پیِر پائولو ایپولیتو**

پل از جلسهٔ گذشته

# جلسهٔ اول: کارخانه را ساختیم

### آنچه آموختیم

محصول اصلی کار شما خودِ کد نیست، بلکه سامانه‌ای است که کد را تولید می‌کند. و «مهار (harness)» همهٔ چیزی است که مدل را احاطه می‌کند.

### آنچه امروز می‌آموزیم

اگر مهندسی عامل‌محور (Agentic engineering) کفِ کارخانه است، پروتکل‌های امروز **استانداردهای صنعتی** آن‌اند: پیچ‌ومهره، قالب داده و کانال ارتباطی.

**جملهٔ کلیدی جلسه:** نسل بعدی نرم‌افزار نوشته نمی‌شود؛ توسط عامل‌های (agent) تعامل‌پذیر **کارگردانی** می‌شود.

# بدون پروتکل باز، هر عامل یک ماشین سفارشی در گاراژ است

- برای هر ابزار (tool) و هر API باید پوششی (wrapper) شکننده و اختصاصی بنویسید.
  و بعد تا ابد نگه‌داری‌اش کنید و با تغییرات بالادست هماهنگش کنید.
- هر API می‌شود «استانداردی برای یک نفر» — یعنی بدهی فنی خالص.
- ساعت‌ها زمان و توکن‌های ارزشمندتان صرف کارهای کم‌بازده می‌شود و در نقش رهبر ارکستر (Conductor) حبس می‌مانید.

**با پذیرش لایه‌های تعامل‌پذیر استاندارد:** مهار عامل شما به سکویی ماژولار و «وصل‌کن و کار کن» بدل می‌شود. وقت کمتری صرف دیباگ (debug) بارهای JSON سفارشی می‌کنید و بیشتر به هدایت قصد (intent) در سطح بالا می‌پردازید.

# پنج استاندارد، پنج نقش در کارخانه

### MCP — پریز USB-C

مدل را بی‌واسطه به پایگاه داده، فایل‌سیستم و APIهای وب وصل می‌کند.

### A2A — بی‌سیم کارخانه

عامل‌های متخصص با آن مذاکره می‌کنند، هم‌فکری می‌کنند و کار را به هم می‌سپارند.

### A2UI — ویترین مولد

خروجی خام و پیچیده را به مؤلفه‌های بصری امن و تعاملی بدل می‌کند.

### AP2 و UCP — شبکهٔ تأمین و تراکنش

مذاکره و اجرای ایمن تراکنش‌های تجاری خودگردان (autonomous).

### مهارت‌ها — دفترچهٔ راهنما

دستورهای (instruction) ساده و اسکریپت‌هایی که در محیط سندباکس (sandbox) اجرا می‌شوند. موضوع کامل جلسهٔ سوم.

# نقشهٔ امروز

**۱. MCP**
عامل چگونه ابزار می‌گیرد

←

**۲. معماری**
از یکپارچه به تخصصی

←

**۳. A2A**
عامل چگونه با عامل حرف می‌زند

←

**۴. A2UI**
عامل چگونه با انسان حرف می‌زند

←

**۵. تجارت**
عامل چگونه پول خرج می‌کند

**تمرکز این مقاله:** نه ساختن سرور، بلکه **مصرف کردن**. کدنویس شهودی نمی‌خواهد ساعت‌ها پیکربندی سرور بنویسد؛ می‌خواهد به فهرست‌های موجود وصل شود و فوری ابرقدرت بگیرد.

بخش یک

# MCP از نگاه کدنویس شهودی

کشف، پیکربندی، اتصال — و بس.

# سیم‌کشی دستی در برابر سوکت استاندارد

### روش سنتی سازمان‌ها

هر اتصال تازه یعنی: پوشش (wrapper) اختصاصی REST، کلید API، سیاست (policy) تازه‌سازی توکن، و تجزیه‌گر JSON سفارشی.

### با MCP

یک سوکت استاندارد همهٔ آن دردسرها را کنار می‌گذارد. ابزار را وصل می‌کنید و تمام.

**تغییر ذهنیت:** برای کدنویس شهودی، اولویت **مصرف** است نه **ساخت**.

# سه گام ورود یک سرور MCP

**۱. کشف**
سرور آماده را پیدا کن

←

**۲. پیکربندی**
دامنه، مجوز و اعتبارنامه (credential) را تعیین کن

←

**۳. اتصال**
دست بده و فهرست ابزارها را بگیر

**توجه:** در هیچ‌کدام از این سه گام، کد یکپارچه‌سازی نمی‌نویسید. اگر دارید کد می‌نویسید، احتمالاً راه را اشتباه رفته‌اید.

# گام ۱ – کشف: سه منبع

### فهرست‌های عمومی

صدها سرور آماده. عالی برای نمونه‌سازی سریع محلی، اما **بازبینی‌نشده** – مسئولیتش با شماست.

### سرورهای معتبر شخص ثالث

سکوهای بازبینی‌شده با نقاط پایانی میزبانی‌شده؛ مثل سرورهای رسمی برای سرویس‌های مدیریت‌شده.

### فهرست‌های داخلی سازمان

ابزارهای داخلی به‌شکل سرور MCP عرضه و در فهرستی امن ثبت می‌شوند؛ معمولاً پشت دروازهٔ API.

**نکتهٔ کاربردی:** امنیت اولویت نخست است. تا جای ممکن سراغ دستورالعمل رسمی بروید و **هرگز** به سرور عمومی جامعه اعتبارنامه ندهید.

# گام ۲ – پیکربندی

- **پیش‌نیازها را بررسی کنید:** سرور به چه رانتایم، نسخه یا سرویسی وابسته است؟
- **دامنه و معیار دسترسی را مشخص کنید:** عامل به کدام پوشه، کدام پروژه و کدام جدول‌ها دسترسی دارد؟
- **مجوز خواندن/نوشتن را تعریف کنید:** پیش‌فرض را فقط‌خواندنی بگذارید و نوشتن را استثنا کنید.
- **احراز هویت (identity):** کلیدها و توکن‌ها (token) از راه فایل محیطی، نه درون پرامپت یا اسکریپت.

**ضدالگوی رایج:** چسباندن کلید `API` داخل دستور عامل. این کلید در گزارش‌ها، تاریخچهٔ نشست (session) و احتمالاً در زمینهٔ مدل باقی می‌ماند.

# گام ۳ — اتصال و تأیید

پس از پیکربندی، میزبان اتصال را برقرار می‌کند. برای اطمینان، یک درخواست دست‌دادن ساده بفرستید که ابزارهای موجود را فهرست کند و اسکیمای خروجی را اعتبارسنجی کند.

**نشانهٔ سلامت**

فهرست ابزارها با نام، توضیح و اسکیمای پارامترها برمی‌گردد و عامل می‌تواند همان‌ها را فرا بخواند.

**نشانهٔ خرابی**

فهرست خالی است، اسکیما (schema) ناقص است، یا عامل نام ابزاری را می‌سازد که در فهرست نیست.

# مسئلهٔ N×M در نمونه‌سازی

فرض کنید با N مدل مختلف کار می‌کنید و می‌خواهید به M ابزار بیرونی وصل‌شان کنید. توسعهٔ سنتی برای هر تقاطع مدل و ابزار کد یکپارچه‌سازی جدا می‌خواهد.

**5 × 10 = 50**

پنج مدل و ده ابزار یعنی نگه‌داری پنجاه نقطهٔ یکپارچه‌سازی اختصاصی. تغییر یک API، چند حلقهٔ تجزیه (decomposition) را هم‌زمان می‌شکند.

**5 + 10 = 15**

با پروتکل استاندارد، پیچیدگی از ضربی به جمعی می‌افتد: هر مدل یک‌بار به MCP وصل می‌شود و هر ابزار یک‌بار.

**به‌زبان مهندسی:** از O(N × M) به O(N + M).

# چرا این برای شما مهم است

با استانداردشدن تعریف ابزارها، حامل‌های (Transport) استاندارد را بدون نوشتن کد یکپارچه‌سازی مستقیم به مهار عامل وصل می‌کنید.

| حامل | چطور کار می‌کند | کِی مناسب است |
| --- | --- | --- |
| `stdio` | میزبان، سرور را به‌عنوان یک زیرفرایند محلی اجرا می‌کند و پیام‌های `JSON-RPC 2.0` را از راه ورودی و خروجی استاندارد رد و بدل می‌کند. | کار محلی و نمونه‌سازی؛ بدون نیاز به پیکربندی شبکه. |
| **SSE روی HTTP** | میزبان به یک نقطهٔ پایانی راه دور وصل می‌شود و داده به‌صورت زمان‌واقعی جاری می‌شود. | وابستگی (dependency) کمتر، همیشه به‌روز، ردپای کوچک‌تر و چرخهٔ عمر ساده‌تر — اما بار بیشتری روی سرور ابری می‌گذارد. |

کارگاه ۱ — ۲۰ دقیقه

# نخستین اتصال MCP خودتان

۱

### کشف

یک سرور رسمی برای فایل‌سیستم یا جست‌وجو انتخاب کنید.

۲

### پیکربندی

دسترسی را به یک پوشهٔ آزمایشی محدود کنید؛ اعتبارنامه فقط از متغیر محیطی.

۳

### اتصال

دست بدهید، فهرست ابزارها را بگیرید، اسکیماها را بخوانید.

۴

### تست

کاری بدهید که عامل ناچار شود از ابزار استفاده کند و مسیر (trajectory) فراخوان را دنبال کنید.

**معیار پذیرش:** تلاش برای خواندن بیرون از پوشهٔ مجاز باید **رد شود**. اگر رد نشد، پیکربندی‌تان امن نیست.

بخش دو

# وقتی خراب می‌شود

عامل پارامتر توهم (hallucination) می‌زند، ابزار اشتباه را صدا می‌کند، یا بار داده را نمی‌تواند تجزیه کند. حالا چه؟

# لوله‌ها را بازرسی کنید، نه سیستم پرامپت را

### MCP Inspector

پنل وب محلی که با آن هر سرور محلی یا راه دور را دستی پرس‌وجو می‌کنید: دیدن اسکیمای ابزارهای فعال، آزمودن دستی ورودی‌ها، و بازرسی بسته‌های خام JSON-RPC 2.0 — بدون آنکه گردش‌کار (workflow) اصلی عامل را راه بیندازید.

### Chrome DevTools

مناسب محیط‌های توسعهٔ وب‌محور و دیباگ اتصال‌های SSE: ردیابی جریان‌های ورودی و بررسی تأخیر (latency) سرور.

**عادتی که باید جا بیفتد:** وقتی عامل فراخوان اشتباه می‌زند، وسوسه می‌شوید سیستم پرامپت را دست‌کاری کنید. این کار وقت تلف‌کردن است. اول دادهٔ خام انتقال را ببینید.

# بایدها — جعبه‌ابزار مصرف امن

- **سرورهای عمومی را پیش از اتصال ممیزی کنید.**

  به‌ویژه اگر قرار است به فایل‌سیستم یا اعتبارنامه‌های محلی شما دسترسی داشته باشند.

- **برای ابزارها هم از بازیابی (retrieval) پویا (dynamic) استفاده کنید.**

  ابزار را فقط هنگام نیاز از فهرست بارگذاری کنید و پس از پایان کار از زمینه (context) بیرون بیندازید تا توجه رقیق نشود.

- **از دروازه‌ها (gate) و فهرست‌های داخلی استفاده کنید.**

  یعنی مصرف اسکیماهای تأییدشده و حکمرانی‌شده، نه اجرای کد بازبینی‌نشده.

- **انسان را در حلقه (loop) نگه دارید.**

  ورودی ابزار را پیش از فراخوان به کاربر نشان دهید تا خروج تصادفی یا مخربِ داده رخ ندهد.

- **استفاده از ابزارها را برای ممیزی ثبت کنید.**

# نبایدها — دام‌های رایج

- **اگر می‌شود مصرف کرد، نسازید.**

  پیش از نوشتن پوشش REST اختصاصی، دنبال سرور MCP موجود بگردید.

- **سرور عمومی و تأییدنشده را در تولید به کار نگیرید.**

  برای نمونهٔ آخر هفته عالی‌اند؛ برای منطق اصلی کسب‌وکار خطرناک.

- **اعتبارنامه را سخت‌کد نکنید.**

  هرگز کلید یا توکن را در پرامپت عامل یا اسکریپت محلی نچسبانید.

- **به محیط تولید وصل نشوید.**

  از پروژهٔ توسعه با دادهٔ غیرتولیدی یا مبهم‌سازی‌شده استفاده کنید.

- **برای به‌روزرسانی استفاده نکنید.**

  اگر ناچارید به دادهٔ واقعی وصل شوید، سرور را فقط‌خواندنی کنید.

- **دسترسی گسترده به همهٔ پروژه‌ها ندهید.**

  دامنه را به یک پروژه و منابع همان پروژه محدود کنید.

اگر امروز فقط یک چیز از این بخش با خود ببرید: دامنهٔ دسترسی را کوچک کنید، اعتبارنامه را از متن جدا کنید، و پیش‌فرض را فقط‌خواندنی بگذارید.

سه قاعده‌ای که بیشتر حوادث امنیتی این حوزه را پیشگیری می‌کنند

بخش سه

# تکامل معماری‌های عامل‌محور

ساختن ابزار کافی نیست؛ باید بدانیم عامل‌ها چطور باید سازمان‌دهی شوند.

# یک الگوی تکرارشونده در تاریخ فناوری

گذار از پیکربندی دستی و سطح‌پایین، به هماهنگی اعلانی و مبتنی بر قصد — جایی که کاربر نمی‌گوید چگونه بساز، می‌گوید چه می‌خواهم.

### زیرساخت

از پیکربندی دستی سرورها به «زیرساخت به‌مثابه کد».

### یادگیری ماشین

از طراحی دستی شبکه‌های عصبی به AutoML: «شبکه‌های عصبی، شبکهٔ عصبی طراحی کنند.»

### نرم‌افزار

از نوشتن نحو به کدنویسی شهودیِ کل برنامه.

**اما:** خودِ معماری عامل‌محوری که این برنامه‌ها را می‌سازد، دارد مسیر دیگری از تاریخ نرم‌افزار را تکرار می‌کند: گذار از یکپارچه به میکروسرویس.

# «چاقوی همه‌کاره»: عامل یکپارچه

همان‌طور که برنامه‌های وب اولیه یک کدبیس غول‌آسا بودند که صورت‌حساب، رابط کاربری و پایگاه داده در آن به هم چسبیده بودند، بیشتر پروژه‌های کدنویسی شهودی (Vibe coding) هم به یک عامل یگانه تکیه می‌کنند: یک پرامپت (prompt) پیچیده، ده‌ها ابزار، و یک عامل که همهٔ کلاه‌ها را سر می‌گذارد.

**و این بد نیست:** با همین معماری می‌توان در یک آخر هفته یک نمونهٔ اولیهٔ کارآمد سرِهم کرد. مشکل وقتی شروع می‌شود که بخواهید مقیاس بگیرید.

# سقف معماری یکپارچه

### اصطکاک مقیاس

نمی‌توانید «منطق پایگاه داده» را بهینه کنید بی‌آنکه «منطق رابط کاربری» را که در همان پرامپت قرار دارد گیج کنید. هرچه ابزارها بیشتر، تصمیم‌گیری بدتر: فضای جست‌وجوی کنش بعدی بیش‌ازحد بزرگ می‌شود.

### اضافه‌بار زمینه

چپاندن دستورهای کلان، ده‌ها اسکیمای پیچیدهٔ ابزار و تاریخچهٔ گفت‌وگو در یک پرامپت، حافظهٔ کاری مدل را پر می‌کند.

### نقطهٔ شکست یکتا

یک اشکال در یک ابزار یا دستور، کل عامل را به توهم یا سقوط می‌کشاند. دادهٔ نامربوط یا خراب در زمینه می‌ماند و استدلال‌های بعدی را هم می‌شکند.

**همان درسی که مهندسان یادگیری ماشین سال‌ها پیش گرفتند:** وقتی مدل‌های `AutoML` ارزش کسب‌وکاری‌شان را ثابت کردند، تیم‌ها آن جعبه‌سیاه‌ها را در قلب کسب‌وکار رها نکردند؛ آن‌ها را به مراحل قابل‌مشاهده و قابل‌نگه‌داری. **تخصص، سازوکار مقیاس است.**

# راه‌حل اول: تخصص‌گرایی داخلی

افراز منطقی یک عامل یکپارچه (Single-agent monolith) به زیرعامل‌های (subagent) هدفمند. هر زیرعامل یک سیستم پرامپت بسیار متمرکز دارد و فقط زیرمجموعه‌ای مرتبط از ابزارها.

**هنوز یکپارچه است:** این زیرعامل‌ها از مرز شبکه عبور نمی‌کنند؛ همان رانتایم (runtime) و همان حافظهٔ زیرین را به اشتراک می‌گذارند. یعنی تعریف‌های پیچیده ماژولار می‌شوند، در حالی که تأخیر پایین و مدیریت سادهٔ وضعیتِ یک برنامهٔ تک‌فرایندی حفظ می‌شود.

# سه بهبود فنی که تخصص‌گرایی می‌آورد

۱

### کوچک‌شدن فضای جست‌وجو

با محدودکردن ابزارها و مهارت‌های (skill) هر زیرعامل (مثلاً دادن فقط ابزار پرس‌وجو به عامل پایگاه داده)، خطای فراخوان ابزار (tool call) و توهم به‌شدت کم می‌شود.

۲

### مهار رقیق‌شدن توجه

مدل زیرین می‌تواند روی یک دامنهٔ واحد تمرکز کند و استدلالش تیزتر می‌شود.

۳

### بهینه‌شدن بار زمینه‌ای

هماهنگ‌کننده (Orchestrator) به‌جای پردازش کل درخت منطق، فقط کارها را مسیریابی (routing) می‌کند؛ پس نسبت سیگنال به نویز در کانتکست ویندوی زیرعامل‌ها بالا می‌ماند.

# راه‌حل دوم: معماری توزیع‌شدهٔ چندعاملی

رهبران صنعت از عرضهٔ صرفِ API فراتر رفته‌اند و عامل‌های تخصصی دامنه‌ای منتشر می‌کنند که در اکوسیستم اختصاصی خودشان با تسلط کامل کار می‌کنند.

### بسازیم؟ هزینهٔ نگه‌داری

ساختن زیرعامل اختصاصی برای هر سکوی شخص ثالث یعنی شما مسئول به‌روزرسانی منطق پرامپت و تعریف ابزارها در برابر هر تغییر محصول و اسکیمای بالادست می‌شوید.

### بخریم؟ عامل متخصص رسمی

این عامل‌ها را تیم‌هایی نگه می‌دارند که در آن دامنه تخصص عمیق دارند. با واگذاری (handoff) نگه‌داری منطق تخصصی، هماهنگ‌کننده — و شما — تمام تمرکزتان روی ارزش یکتای برنامهٔ خودتان می‌ماند.

**اما گلوگاه تازه‌ای پیدا می‌شود:** پراکندگی. عامل یکی با ADK نوشته شده، دیگری روی LangChain، سومی چیزی کاملاً سفارشی. زبان‌های متفاوت، ساختار بار متفاوت، مدیریت وضعیت متفاوت.

بخش چهار

# A2A: زبان مشترک نیروی کار مجازی

همان‌طور که HTTP وب را استاندارد کرد، A2A نیروی کار مجازی را استاندارد می‌کند.

# A2A دقیقاً چه چیزی را حل می‌کند؟

اگر برای هر متخصصی که می‌خواهید «استخدام» کنید ناچار باشید کد یکپارچه‌سازی سفارشی و حلقهٔ تصحیح خطای اختصاصی بنویسید، مفهوم «تیم مجازی» فوری به یک پروژهٔ یکپارچه‌سازی بدل می‌شود و هزینهٔ نگه‌داریِ آن پل‌های سفارشی، کل پروژه را می‌بلعد.

- لایهٔ ارتباطی جهانی برای سامانه‌های عامل‌محور؛ ابتدا در گوگل ساخته شد و سپس به بنیاد لینوکس اهدا شد.
- تفاوت‌های حامل شبکه، چارچوب زیرین، زبان برنامه‌نویسی و ساختار بار را انتزاع می‌کند.
- هماهنگ‌کنندهٔ مرکزی می‌تواند هر متخصصی در اکوسیستم را کشف، سوار و با او همکاری کند — کاملاً بی‌اعتنا به اینکه آن متخصص زیر پوسته چطور ساخته شده.

# چرا متخصص را مثل ابزار نمی‌شود صدا زد؟

صاحب‌خانه‌ای که آشپزخانه‌اش را بازسازی می‌کند، دو انتخاب دارد: ابزار و دفترچهٔ راهنما بخرد و خودش بسازد، یا کار را به متخصصی بسپارد که کارش ساختن آشپزخانه است.

### ابزار

ساز منفعل. یک درخواست کامل و بی‌نقص می‌گیرد و یک پاسخ می‌دهد. «بفرست و فراموش کن».

### متخصص

شریک همکار. وقتی به حالت مرزی (edge case) یا اشتباهی در طرح اولیهٔ شما برخورد، مکث می‌کند، دربارهٔ هزینه/فایده‌ها با شما مشورت می‌کند و بعد ادامه می‌دهد.

**پرسش تصمیم‌گیری:** «آیا فراخوان‌کننده فقط یک **نتیجه** می‌خواهد، یا می‌خواهد یک **شرکت‌کنندهٔ دیگر مسئولیت** را بر عهده بگیرد؟»

# دامنهٔ کران‌دار در برابر بی‌کران — و مسئلهٔ GOTO

### ابزار: دامنهٔ کران‌دار

ورودی کاملاً قالب‌بندی‌شده، خروجی مشخص، جریان کنترل قابل‌پیش‌بینی.

### عامل: دامنهٔ بی‌کران

محیط واقعی پر از ساختار دادهٔ مبهم، نیازمندی گمراه‌کننده و ترجیح‌های متناقض است. به‌ندرت می‌توان همهٔ جزئیات یک گردش‌کار پیچیده را بدون گفت‌وگوی چندنوبتی مشخص کرد.

**مسئلهٔ GOTO:** اگر عاملِ همکار را در پوشش یک ابزار استاندارد بچپانید، معادل معماریِ دستور `GOTO` را وارد هماهنگ‌کننده کرده‌اید: جریان کنترل از زمینهٔ ساختارمند خارج می‌شود، عامل به حالت وقفه می‌رود، اطلاعات بیشتری می‌خواهد و ممکن است هرگز خروجی مورد انتظار را برنگرداند.

**راه‌حل:** این مسیریابی همکارانه را در لایهٔ `A2A` ایزوله کنید تا لایهٔ ابزار (`MCP`) تمیز، قابل‌پیش‌بینی و کاملاً ساختارمند بماند.

# کارت عامل: رزومهٔ ماشین‌خوان

برای آنکه کشف ممکن شود، هر عامل با یک کارت عامل (Agent Card) تعریف می‌شود: هویتی ماشین‌خوان که سه چیز را مشخص می‌کند.

### توانایی‌ها

چه کارهایی را می‌تواند انجام دهد.

### امنیت و انطباق

سیاست‌های مدیریت داده و مجوزهای لازم.

### اسکیمای تعامل

عامل‌های دیگر چگونه باید با آن ارتباط بگیرند.

**پیامد اقتصادی:** متخصصی که در یک گوشهٔ جهان با کدنویسی شهودی ساخته شده، می‌تواند به‌دست هماهنگ‌کننده‌ای در گوشهٔ دیگر جهان کشف و «استخدام» شود.

# فهرست‌های ثبت: از مارکت‌پلیس عمومی تا سازمان خصوصی

### فهرست عمومی (مارکت‌پلیس)

مثل یک آژانس استعدادیابی جهانی: عامل‌های متخصص خود را برای کشف‌شدن به جهان عرضه می‌کنید و تخصصتان را به هزاران هماهنگ‌کنندهٔ دیگر واگذار می‌کنید.

### فهرست خصوصی (سازمانی)

محیطی امن و حکمرانی‌شده: توسعه‌دهنده‌ای در یک شرکت بزرگ، عاملی می‌سازد که گردش‌کاری داخلی را خودکار می‌کند و آن را برای استفادهٔ دیگر بخش‌ها ثبت می‌کند — بدون آنکه امنیت به خطر بیفتد.

**مدل کسب‌وکار تازه:** عامل به‌مثابه سرویس. مارکت‌پلیس‌های (marketplace) ابری بخش سخت — صورت‌حساب — را خودکار می‌کنند؛ و برای ریزتراکنش‌های ماشین‌به‌ماشین، الگوهایی مانند بازگرداندن کد وضعیت `HTTP 402` با صورت‌حساب ماشین‌خوان و اثبات رمزنگاشتی پرداخت وجود دارد.

# پیاده‌سازی: سمت عرضه و سمت تقاضا

### عرضه — عامل خود را A2A کنید

۱) کارت عامل را تعریف کنید. ۲) مجری عامل را بسازید (لایهٔ ترجمه که درخواست‌های A2A را به فراخوان‌های چارچوب زیرین تبدیل می‌کند). ۳) نقطهٔ پایانی سازگار با A2A را منتشر کنید.

### تقاضا — عامل راه دور را وصل کنید

یا اتصال مستقیم نقطه‌به‌نقطه، یا کشف غیرمستقیم از راه فهرست ثبت (Registry).

```
# 1. Direct instantiation via a hardcoded endpoint
billing_specialist = RemoteA2aAgent(
    name="billing_agent",
    endpoint="https://api.vendor.com/v1/billing/a2a",
)

# 2. Indirect instantiation via an Agent Registry
registry = AgentRegistry(project_id=project_id, location=location)
agent_name = f"projects/{project_id}/locations/{location}/agents/YOUR_AGENT_ID"
my_remote_agent = registry.get_remote_a2a_agent(agent_name=agent_name)
```

بخش پنج

# A2UI: وقتی خروجی عامل، رابط کاربری است

همکار شما در پاسخ به «عملکرد فصل چهارم به تفکیک منطقه چطور بود؟» صفحه‌گسترده تحویلتان نمی‌دهد؛ نموداری روی تخته می‌کشد.

# شکاف ارتباطی

### آنچه امروز می‌گیریم

عامل JSON خام برمی‌گرداند. بعد خودتان نمودار را می‌سازید: وارد کردن کتابخانه، پیکربندی محورها، مدیریت وضعیت.

### آنچه می‌خواهیم

عامل رابط کاربری کامل و تعاملی تولید کند، نه فقط تودهٔ داده.

**هزینهٔ پنهان:** این جابه‌جایی مدام میان «پرسیدن برای بینش» و «انجام‌دادن کار رابط کاربری»، جریان کدنویسی شهودی را می‌شکند.

# رابط کاربری مولد چیست؟

مدل‌های زبانی در رانتایم و بر پایهٔ قصد و زمینهٔ کاربر، رابط می‌سازند. به‌جای آنکه توسعه‌دهنده هر حالت ممکن رابط را سخت‌کد کند، مدل رابط مناسب را به‌اقتضا تولید می‌کند.

وقتی می‌پرسید «فروش فصل چهارم را به تفکیک منطقه مقایسه کن»، سامانه فقط داده بازیابی نمی‌کند: چیدمانی تعاملی، کارت‌ها، صافی‌ها و کنترل‌های متناسب با همان خواسته را می‌سازد.

**اما چالش، ایمنی است:** اجرای کد دلخواهِ تولیدشده توسط مدل زبانی یعنی تزریق کد، حملهٔ XSS و عوارض جانبی کنترل‌نشده.

# چرا A2UI امن است: نُت موسیقی، نه ضبط صدا

آهنگ‌ساز به نوازنده‌ها فایل صوتی نمی‌دهد؛ نُت می‌دهد. همان نُت روی پیانو، ارکستر یا سینتی‌سایزر نواخته می‌شود و هر ساز با صدای خودش اجرایش می‌کند.

**عامل**
قصد را می‌نویسد: چه چیزی رندر شود و چطور ترکیب شود

←

**کاتالوگ (Catalog)**
فهرست مؤلفه‌های مورد اعتماد را تعیین می‌کند

←

**رندرگر**
با کتابخانهٔ مؤلفهٔ خودش بومی اجرا می‌کند

**مدل امنیتی در یک جمله:** عامل کد اجرایی تولید نمی‌کند و پیکسل از پیش رندرشده هم نمی‌فرستد؛ فقط از کاتالوگی که کلاینت به آن اعتماد دارد **درخواست مؤلفه** می‌کند. مثل لِگو، اما قطعه‌ها مؤلفه‌های سامانهٔ طراحی خودِ شمایند.

# کاتالوگ پایه و پیام A2UI

| دسته | مؤلفه‌ها |
|---|---|
| **چیدمان** | `Row, Column, List` |
| **نمایش** | `Text, Image, Icon, Divider` |
| **ظرف** | `Card, Modal, Tabs` |
| **رسانه** | `Video, AudioPlayer` |
| **تعاملی** | `Button, TextField, CheckBox, Slider, DateTimeInput, ChoicePicker` |

**«پایه» نه «استاندارد»:** این نام‌گذاری عمدی است — رابط‌های تولیدی باید **کاتالوگ خودشان** را بیاورند و مؤلفه‌های موجودشان را به انواع `A2UI` نگاشت کنند. عامل تغییر نمی‌کند؛ فقط نگاشت کاتالوگِ رندرگر عوض می‌شود.

# دو الگوی تولید A2UI

### الگوی پیش‌فرض: مدل، رابط را می‌سازد

مدل مالک چیدمان است و آن را با قصد کاربر تطبیق می‌دهد. همان عامل برای «این منطقه‌ها را مقایسه کن» و «روندها را نشان بده» دو رابط متفاوت می‌سازد.

### تخصصی‌شده: ابزار، ساختار ثابت برمی‌گرداند

یک فراخوان ابزار، بدون صرف توکن برای ساخت رابط، با خروجی کاملاً قابل‌پیش‌بینی. ابزار در عمل یک قالب سمت سرور است.

| پرسش کاربر | خروجی مناسب |
| --- | --- |
| «میانگین چقدر است؟» | داده به‌صورت متن |
| «این منطقه‌ها را مقایسه کن» | رابط تولیدشده به‌دست مدل |
| «داشبوردم را نشان بده» | رابط ساخته‌شده به‌دست ابزار (قالب ثابت) |
| تماس برنامه به برنامه | دادهٔ JSON |

# بوم تعاملی و خروجی دوگانه

### بوم

رابط‌های گفت‌وگومحور سنتی خطی‌اند و هر پاسخ ایستاست. بوم یک فضای کاری ماندگار می‌سازد که هم عامل و هم کاربر می‌توانند ویرایشش کنند — سندی زنده به‌جای تاریخچهٔ گفت‌وگو.

### خروجی دوگانه

هم داده بدهید، هم رابط: کلاینت‌های برنامه‌ای فیلد `ui` را نادیده می‌گیرند و از داده استفاده می‌کنند؛ کلاینت‌های انسانی رابط را رندر می‌کنند.

**در تولید:** خروجی مدل احتمالاتی است. همیشه اعتبارسنجی اسکیما بگذارید، در صورت خطا دوباره تلاش کنید، و در نهایت به پاسخ متنی عقب‌نشینی کنید. رندرگر هرگز نباید بار بدشکل ببیند.

بخش شش

# عامل‌ها و تجارت

تا اینجا تعامل‌ها عمدتاً «خواندن» بودند. حالا عامل باید کنشی انجام دهد که پیامد مالی واقعی دارد.

# UCP — مترجم جهانی خرید

ساعت دو بامداد است و گرسنه‌اید. دستیار هوشمندتان را می‌فرستید غذا سفارش دهد.

**بدون UCP**

هوش مصنوعی باید مرورگر را باز کند، به وب‌سایت بدطراحی رستوران برود، تیک «گواکامولهٔ اضافه» را پیدا کند و امیدوار باشد سایت از کار نیفتد.

**با UCP**

هر رستوران منو، ساعت کاری و گزینه‌های سفارشی‌سازی را به زبان ماشینی استاندارد منتشر می‌کند. عامل می‌پرسد «هنوز بازید؟ بوریتوی گیاهی دارید؟» و پاسخ ساختارمند می‌گیرد: مالیات، هزینهٔ ارسال و زمان تحویل.

**خلاصه:** UCP همان زبانی است که عامل شما با فروشگاه حرف می‌زند، گزینه‌ها را می‌بیند و سفارش را می‌سازد.

# AP2 — کارت اعتباری با قواعد سخت

غذا در سبد است، اما عامل باید پرداخت کند. بدیهی است که شمارهٔ کارت واقعی‌تان را در پرامپت تایپ نمی‌کنید و نمی‌گویید «هر کاری خواستی بکن».

### گاردریل (مجوزنامه)

پیش از فرستادن عامل، یک قاعدهٔ دیجیتال را تأیید می‌کنید: «تا سقف ۲۵ دلار در این فروشگاه.»

### دست‌دادن

هنگام تسویه، شمارهٔ کارت نشان داده نمی‌شود؛ یک «سفتهٔ» رمزنگاشته و امضاشده ارائه می‌شود: «انسانِ من این سفارش ۱۸٫۵ دلاری را تأیید کرده.» بانک فروشنده فوری امضا را راستی‌آزمایی می‌کند.

### بدون هزینهٔ پنهان

اگر فروشنده بکوشد به‌جای ۱۸٫۵، پنجاه دلار بگیرد، پروتکل فوری جلویش را می‌گیرد، چون قاعده‌ای را که امضا کرده‌اید نقض می‌کند.

# تقسیم کار در یک نگاه

| | UCP | AP2 |
|---|---|---|
| **نقش** | مغزی که تصمیم می‌گیرد چه بخرد، منو را می‌فهمد و سبد را می‌بندد. | کیف پولی که امن پرداخت می‌کند، بی‌آنکه کلاه سرتان برود. |
| **یکپارچگی با** | هر تأمین‌کنندهٔ کسب‌وکار | سامانه‌های پرداخت |
| **ارزش کلیدی** | یکپارچگی واحد، زبان مشترک، معماری توسعه‌پذیر، رویکرد امنیت‌محور | مجوزدهی و ممیزی‌پذیری، اصالت قصد، مهار خطا و توهم عامل، پاسخ‌گویی |

**در یک جمله:** UCP تصمیم می‌گیرد چه چیزی خریده شود؛ AP2 تضمین می‌کند که عامل هرگز به‌اشتباه یک تلویزیون هزار دلاری نخرد.

# جمع‌بندی و تکلیف

**تز جلسه:** با پذیرش استانداردهای بنیادین، بدهی فنی کمرشکنِ یکپارچه‌سازی‌های سفارشی حذف می‌شود و تمرکز شما تماماً به هماهنگ‌کردن منطق باارزش کسب‌وکار منتقل می‌شود — از مکانیکی که APIهای شکننده را سیم‌کشی می‌کند، به معمار یک نیروی کار خودگردان جهانی.

1. دست‌کم دو سرور MCP را به عامل روزمره‌تان اضافه کنید و یک هفته با آن‌ها کار کنید.
   یادداشت کنید کدام فراخوان‌ها ناموفق بودند و ریشهٔ خطا در لایهٔ انتقال چه بود.
2. بایدها و نبایدها را به چند سطر قاعده در `AGENTS.md` تبدیل کنید.
   مثلاً: هرگز به محیط تولید وصل نشو؛ پیش‌فرض همیشه فقط‌خواندنی.
3. یک کارت عامل برای یک متخصص فرضی طراحی کنید.
   توانایی‌ها، سیاست داده، و رفتار در حالت ابهام.

جلسهٔ بعد

# مهارت‌های عامل

نشست، مهارت و حافظه (memory) —
زمینهٔ پویا این‌بار با جزئیات کامل

**پرسشی دارید؟ همین حالا بپرسید.**